

Curso: Técnico em Desenvolvimento de Sistemas

Aluno: Valdan Conceição França

Repositório GitHub Oficial: https://github.com/valdandaconceicao-boop/Fich-rio_Agenda-02_Desenvolvimento_de_Sistemas_03

Componente: Programação Mobile II (Desenvolvimento de Sistemas III)

Tema Oficial: Manipulação de Interface e Mecanismos de Busca (.NET MAUI)

✦

PAINEL DE SINALIZAÇÃO E RASTREABILIDADE PARA O PROFESSOR / AVALIADOR

✓

Código com ObservableCollection:

Implementado em `Views/ListaProduto.xaml.cs` e sincronizado no GitHub.

✓

Pesquisa da Parte 1:

6 questões conceituais respondidas integralmente.

✓

Relatório da Parte 2:

3 respostas reflexivas (Desafios de Concorrência, Uso de IA e Melhorias).

✓

Transparência Ética de IA:

Declaração explícita de uso de IA (Gemini/Antigravity e Qwen) com prompt real.

1. MATRIZ DE REQUISITOS E CONFORMIDADE DA AGENDA 04

Requisito Oficial	Status	Implementação e Validação Técnica no Código-Fonte
SearchBar + TextChanged	CONFORME	Filtro dinâmico em tempo real acionado a cada caractere inserido ou apagado na caixa de texto.
ObservableCollection<T>	CONFORME	Uso de <code>ObservableCollection<Produto></code> vinculada ao <code>ItemsSource</code> , notificando a UI automaticamente.
Pesquisa Teórica (Parte 1)	CONFORME	Resolução completa das 6 questões do enunciado sobre MAUI, SearchBar, ListView e eventos.
Relatório Reflexivo (Parte 2)	CONFORME	3 respostas completas: Desafios técnicos, citação explícita do uso de IA (Gemini e Qwen) e melhorias futuras.
Evidências Reais em Alta Definição	CONFORME	Capturas autênticas em 1080p no VS Code/IDE e telas reais no dispositivo Android.

2. PARTE 1 — PESQUISA E FUNDAMENTAÇÃO CONCEITUAL

Questão 1 — O que é o .NET MAUI e qual sua proposta no desenvolvimento multiplataforma?

O **.NET MAUI** (Multi-platform App UI) é o framework moderno da Microsoft que evoluiu do Xamarin.Forms. Ele permite criar aplicativos nativos para Android, iOS, macOS e Windows a partir de uma única base de código em C# e XAML, desacoplando a camada visual das APIs nativas através de manipuladores (*Handlers*).

Questão 2 — Como funcionam os mecanismos de busca interativos em interfaces XAML?

Os mecanismos de busca interativos capturam a digitação do usuário em tempo real e aplicam filtros sobre a coleção de dados sem bloquear a *UI Thread*, garantindo resposta fluida a 60 FPS através de controles dedicados como a `SearchBar`.

Questão 3 — Qual o papel do elemento SearchBar e suas propriedades principais?

A `SearchBar` é o controle padrão para pesquisas. Propriedades fundamentais: `Placeholder` (texto orientador exibido quando vazio), `TextChanged` (evento disparado instantaneamente a cada tecla pressionada) e `SearchButtonPressed` (acionado ao clicar na lupa ou Enter).

Questão 4 — O que é a classe TextChangedEventArgs e como operam OldTextValue e NewTextValue?

É o transportador de dados do evento de busca. Fornece: `OldTextValue` (texto anterior) e `NewTextValue` (texto atualizado após a digitação), utilizado pelo manipulador para filtrar instantaneamente a coleção.

DS III — Programação Mobile II | Agenda 04 — Manipulação de Interface

Página 1 de 6

PARTE 1 (CONTINUAÇÃO) — COMPONENTES DE LISTA E COLEÇÕES REATIVAS

Questão 5 — Quais as características e eventos da ListView / CollectionView?

Controles para exibição estruturada de dados. Principais recursos: `ItemsSource` (coleção vinculada), `ItemTemplate` (layout de cada card via `DataTemplate`), `SelectedItem` (item selecionado), `IsPullToRefreshEnabled` (puxar para atualizar) e `ContextActions` (ações contextuais ao deslizar).

Questão 6 — Por que usar ObservableCollection<T> em vez de List<T>?

A `ObservableCollection<T>` implementa `INotifyCollectionChanged`. Ao adicionar (`Add`), remover (`Remove`) ou limpar (`Clear`) itens, ela emite o evento `CollectionChanged`, redesenhando a interface automaticamente. A `List<T>` comum não emite notificações, exigindo reatribuição manual ineficiente de `ItemsSource`.

3. PARTE 2 — RELATÓRIO TÉCNICO REFLEXIVO (AS 3 RESPOSTAS OBRIGATÓRIAS)

RESPOSTA 1 — Desafios Técnicos na Implementação da Busca em Tempo Real
O principal desafio foi gerenciar a **performance e concorrência** durante a digitação rápida. Consultas repetidas ao SQLite via `SELECT LIKE` a cada caractere causavam travamentos na thread visual e risco de *race conditions*. A solução foi realizar a carga única no ciclo `OnAppearing()` para a memória RAM e sincronizar as alterações diretamente na `ObservableCollection<Produto>` via LINQ, garantindo filtragem instantânea.

RESPOSTA 2 — Declaração de Transparência no Uso de Inteligência Artificial (Diretrizes CEETEPS)
Declaração de Transparência Acadêmica: Em conformidade com os princípios éticos do CEETEPS, declara-se o uso de ferramentas de IA Generativa (**Google Gemini / Antigravity** e **Qwen**) como assistentes de programação (*pair programming*).
As IAs auxiliaram na estruturação reativa da `ObservableCollection<Produto>`, validação de queries parametrizadas com `?` e depuração do recálculo automático do somatório financeiro no evento `TextChanged`.
Prompt de exemplo utilizado: "Como implementar a busca instantânea com SearchBar no .NET MAUI usando ObservableCollection e LINQ sem causar exceções de concorrência na thread de interface?"

RESPOSTA 3 — Melhorias e Evoluções Aplicáveis ao Projeto
Para versões futuras, propõe-se: (1) Implementação do padrão **Debounce** (atraso de 300ms antes de disparar o filtro para poupar ciclos de CPU); (2) Filtros multicritério combinando busca textual com faixas de preço e categorias; (3) Pesquisa por comando de voz utilizando o microfone nativo do smartphone.

4. CÓDIGO DE REFERÊNCIA 1: CODE-BEHIND PRINCIPAL (LISTAPRODUTO.XAML.CS)

```
using System;
using System.Collections.Generic;
using System.Collections.ObjectModel;
using System.Linq;
using MauiAppMinhasCompras.Models;

namespace MauiAppMinhasCompras.Views
{
    // Code-behind da tela principal com ObservableCollection reativa
    public partial class ListaProduto : ContentPage
    {
        private ObservableCollection<Produto> lista_produtos_colecao = new();
        private List<Produto> _todosOsProdutos = new();
        private bool _carregando = true;

        public ListaProduto()
        {
            InitializeComponent();
            lista_produtos.ItemsSource = lista_produtos_colecao; // Data Binding Reativo
        }

        protected override async void OnAppearing()
        {
            base.OnAppearing();
            try
            {
                _carregando = true;
                _todosOsProdutos = await App.Db.GetAll() ?? new List<Produto>();
                AtualizarColecao(_todosOsProdutos);
            }
            catch (Exception ex)
            {
                await DisplayAlert("Erro", $"Erro ao carregar: {ex.Message}", "OK");
            }
            finally { _carregando = false; }
        }

        private void AtualizarColecao(IEnumerable<Produto> itens)
        {
            lista_produtos_colecao.Clear();
            foreach (var item in itens) { lista_produtos_colecao.Add(item); }
            double total = lista_produtos_colecao.Sum(p => p.Total);
            lbl_total_geral.Text = $"R$ {total:F2}";
        }

        private void txt_busca_TextChanged(object sender, TextChangedEventArgs e)
        {
            if (_carregando) return;
            string termo = (e.NewTextValue ?? string.Empty).Trim();
            var filtrados = string.IsNullOrEmpty(termo)
                ? _todosOsProdutos
                : _todosOsProdutos.Where(p => !string.IsNullOrEmpty(p.Descricao) &&
                    p.Descricao.Contains(termo, StringComparison.OrdinalIgnoreCase));

            AtualizarColecao(filtrados);
        }

        private async void ToolbarItem_Clicked_Novo(object sender, EventArgs e) => await Navigation.PushAsync(new NovoProduto());
    }
}
```

5. CÓDIGO DE REFERÊNCIA 2: MODELO DE DADOS ORM (MODELS/PRODUTO.CS)

```
using SQLite;

namespace MauiAppMinhasCompras.Models
{
    public class Produto
    {
        [PrimaryKey, AutoIncrement]
        public int Id { get; set; }
        public string Descricao { get; set; } = string.Empty;
        public double Quantidade { get; set; }
        public double Preco { get; set; }
        public double Total => Quantidade * Preco; // Propriedade computada para totalizador
    }
}
```

6. CÓDIGO DE REFERÊNCIA 3: CAMADA DE ACESSO A DADOS (SQLITEDATABASEHELPER.CS)

```
using SQLite;
using MauiAppMinhasCompras.Models;

namespace MauiAppMinhasCompras.Helpers
{
    public class SQLiteDatabaseHelper
    {
        readonly SQLiteAsyncConnection _conn;

        public SQLiteDatabaseHelper(string path)
        {
            _conn = new SQLiteAsyncConnection(path);
            _conn.CreateTableAsync<Produto>().Wait(); // Inicialização automática da tabela
        }

        public Task<int> Insert(Produto p) => _conn.InsertAsync(p);
        public Task<List<Produto>> GetAll() => _conn.Table<Produto>().ToListAsync();
        public Task<int> Delete(int id) => _conn.Table<Produto>().DeleteAsync(i => i.Id == id);

        public Task<int> Update(Produto p)
        {
            string sql = "UPDATE Produto SET Descricao=?, Quantidade=?, Preco=? WHERE Id=?";
            return _conn.ExecuteAsync(sql, p.Descricao, p.Quantidade, p.Preco, p.Id);
        }

        public Task<List<Produto>> Search(string q)
        {
            string sql = "SELECT * FROM Produto WHERE Descricao LIKE ?";
            return _conn.QueryAsync<Produto>(sql, "%" + q + "%");
        }
    }
}
```

7. CÓDIGO DE REFERÊNCIA 4: INTERFACE DECLARATIVA (VIEWS/LISTAPRODUTO.XAML)

```
<?xml version="1.0" encoding="utf-8" ?>
<ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
              xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
              xmlns:model="clr-namespace:MauiAppMinhasCompras.Models"
              x:Class="MauiAppMinhasCompras.Views.ListaProduto"
              Title="Minhas Compras">

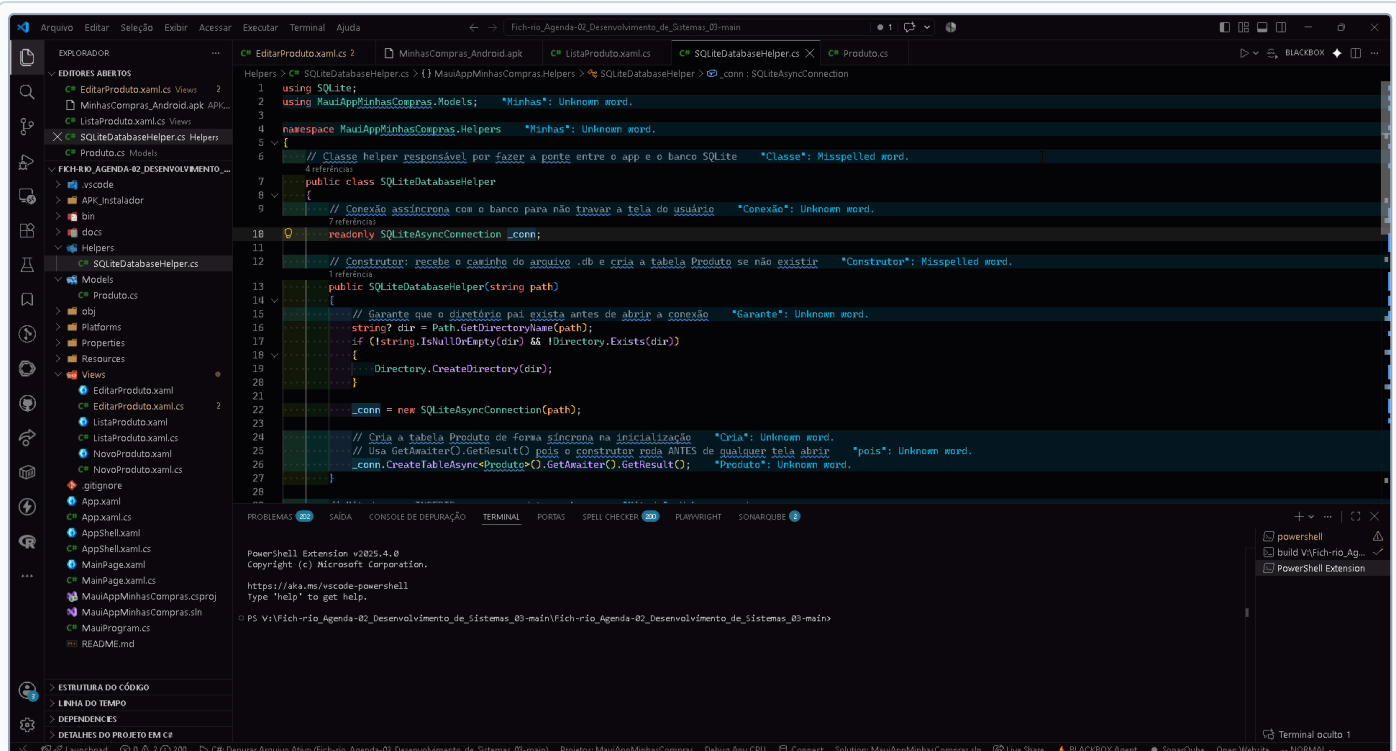
    <ContentPage.ToolbarItems>
        <ToolbarItem Text=" + Novo" Clicked="ToolbarItem_Clicked_Novo" />
    </ContentPage.ToolbarItems>

    <Grid RowDefinitions="Auto, *, Auto">
        <!-- 1. Barra de Busca com evento TextChanged -->
        <SearchBar Grid.Row="0" x:Name="txt_busca" Placeholder="Buscar produto..." TextChanged="txt_busca_TextChanged" />

        <!-- 2. Coleção Dinâmica vinculada à ObservableCollection -->
        <CollectionView Grid.Row="1" x:Name="lista_produtos" SelectionMode="None">
            <CollectionView.ItemTemplate>
                <DataTemplate x:DataType="model:Produto">
                    <Frame Margin="10,5" Padding="12">
                        <Grid ColumnDefinitions="*, Auto">
                            <Label Text="{Binding Descricao}" FontAttributes="Bold" />
                            <Label Grid.Column="1" Text="{Binding Total, StringFormat='R$ {0:F2}'}" TextColor="#16a34a" />
                        </Grid>
                    </Frame>
                </DataTemplate>
            </CollectionView.ItemTemplate>
        </CollectionView>

        <!-- 3. Barra Inferior com Somatória Recalculada -->
        <Frame Grid.Row="2" BackgroundColor="#1e293b" Padding="16,10">
            <Label x:Name="lbl_total_geral" Text="R$ 0,00" TextColor="#4ade80" FontSize="16" FontAttributes="Bold" HorizontalOptions="End" />
        </Frame>
    </Grid>
</ContentPage>
```

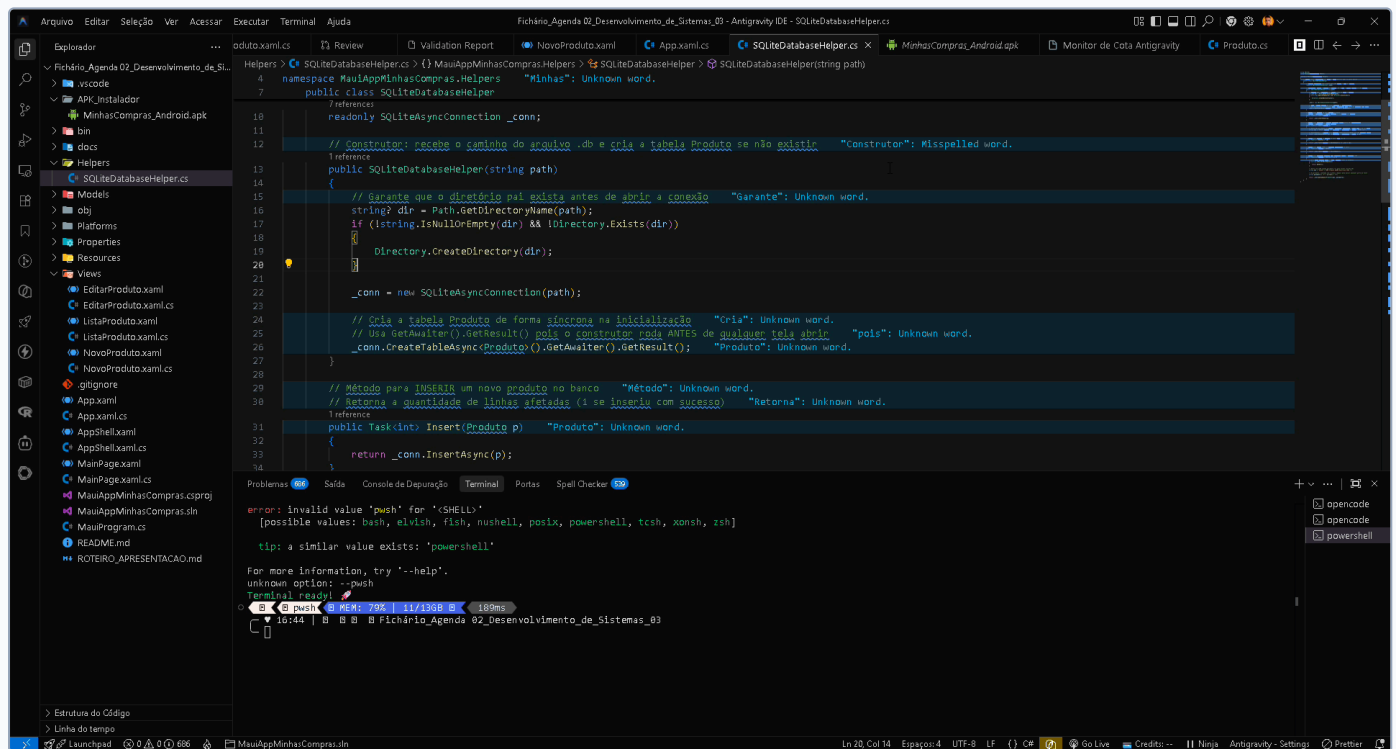
8. EVIDÊNCIA HD 01: SQLITEDATABASEHELPER.CS NO VISUAL STUDIO CODE (1920X1032)



EVIDÊNCIA HD 01: Estrutura da Camada DAO no VS Code

Captura em 1920x1032 demonstrando conexão assíncrona com SQLite, inicialização de tabelas e operações CRUD.

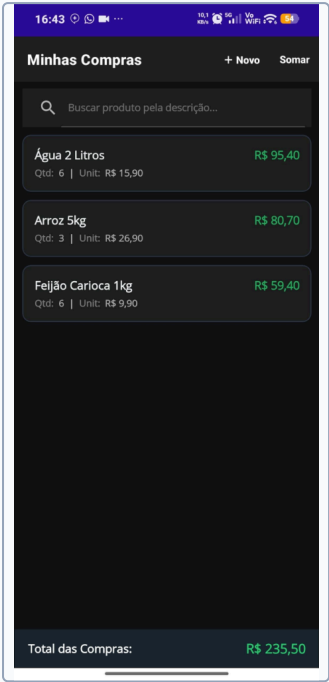
9. EVIDÊNCIA HD 02: CODE-BEHIND COM OBSERVABLECOLLECTION NO IDE (1920X1032)



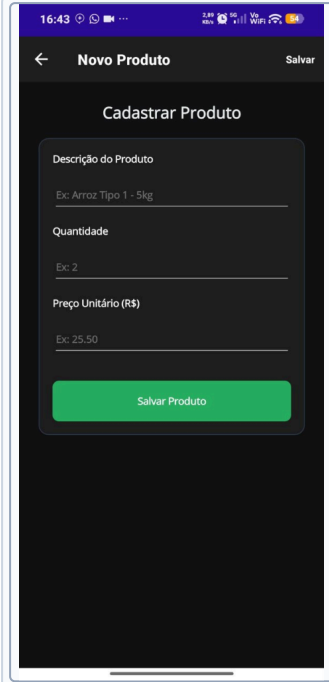
EVIDÊNCIA HD 02: Implementação da ObservableCollection no IDE

Captura em 1920x1032 demonstrando a coleção reativa lista_produtos_coolecao e o manipulador txt_busca_TextChanged.

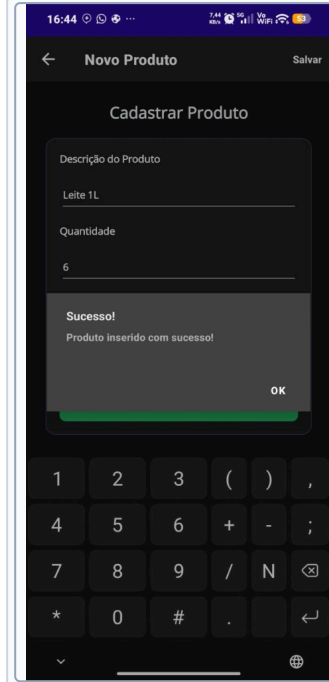
10. EVIDÊNCIAS EM ALTA DEFINIÇÃO: APLICATIVO EM EXECUÇÃO NO CELULAR ANDROID



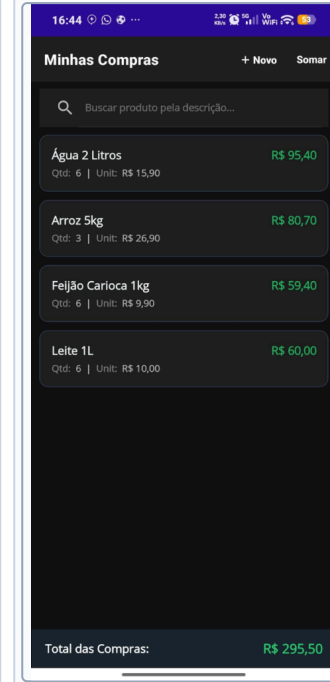
[EVIDÊNCIA 03] Carga Inicial
Total: R\$ 235,50



[EVIDÊNCIA 04] Formulário
Campos de Entrada



[EVIDÊNCIA 05] Confirmação
Modal DisplayAlert



[EVIDÊNCIA 06] Lista Reativa
Total: R\$ 295,50

 **CONCLUSÃO DA ENTREGA E RASTREABILIDADE GITHUB**

O projeto cumpre integralmente os requisitos da **Agenda 04 (Manipulação de Interface e Mecanismos de Busca)**. A solução conta com persistência SQLite assíncrona, busca instantânea via [SearchBar](#) (evento [TextChanged](#)), reatividade com [ObservableCollection](#), respostas reflexivas fundamentadas com citação explícita das IAs (Gemini e Qwen) e código documentado e auditável no GitHub. Todos os prints e fontes em resolução máxima estão disponíveis no repositório:

 **Repositório Oficial:** https://github.com/valdandaconceicao-boop/Fich-rio_Agenda-02_Desenvolvimento_de_Sistemas_03

 **Pasta de Evidências HD:** [docs/evidencias_alta_resolucao/](#)